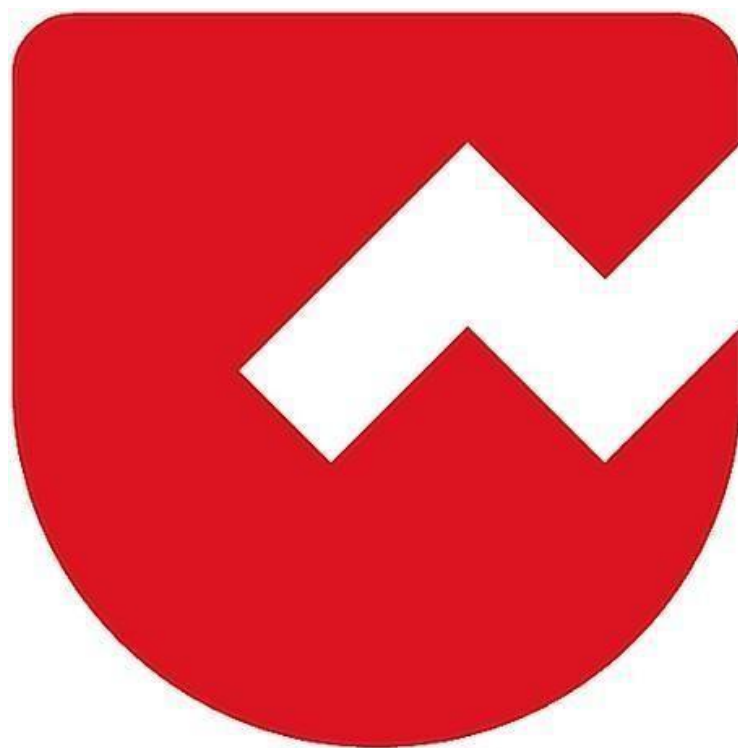




Zoho Schools for Graduate Studies



Notes

Date: 23/10/2025
Day: Thursday

EXCEPTION HANDLING

1. What is an Exception?

- An exception is an unexpected event or error that disrupts the normal flow of instructions in a program.
- It can usually be anticipated and handled by the programmer using specific code structures (like try-catch blocks).
- Common examples are trying to divide by zero or accessing a non-existent file.
- Handling exceptions makes programs more robust and user-friendly, preventing crashes.

2. What is an Error?

- An error is a serious problem in a program, often caused by the environment or system.
- Errors usually cannot be handled or recovered from by the code itself.
- Examples include running out of memory, stack overflow, or hardware failure.
- When an error occurs, the program often must terminate.

3. What is the effect of an Exception?

- When an exception occurs, it interrupts the normal sequence of execution in a program.
- If not caught, the program may abruptly terminate.
- If handled, control can be transferred to code that manages the issue, allowing the program to recover or exit cleanly.
- Unhandled exceptions often generate error messages or logs.

4. What is the impact of Exception?

- Improperly handled exceptions can cause a program to crash, lose data, or behave unexpectedly.
- This can result in poor user experience or loss of trust in software.
- Important processes may be left incomplete, causing system instability.
- Financial/business losses can occur if exceptions shut down critical software.

5. Who gets most affected because of an Exception?

- The primary party affected is the organization that depends on the software.
- Users may experience inconvenience, lost work, or failed transactions.
- The organization may suffer reputational harm, lost revenue, or increased support costs.
- Developers and IT support may also face extra workload fixing and responding to issues.

Exception Example:

```
public class Main {  
    public static void main(String[ ] args) {  
        int[] myNumbers = {1, 2, 3};  
        System.out.println(myNumbers[10]); // error!  
    }  
}
```

The Output:

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 10
at Main.main(Main.java:4)

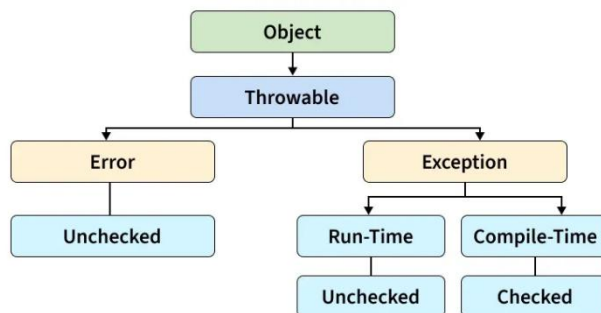
Note: ArrayIndexOutOfBoundsException occurs when you try to access an index number that does not exist.

Java Exception Hierarchy:

Java Exception class is the parent class of all exceptions in Java. If you catch an Exception object, you can be sure that you catch all the exceptions. The Java Exception class has many subclasses that represent the different types of exceptions.

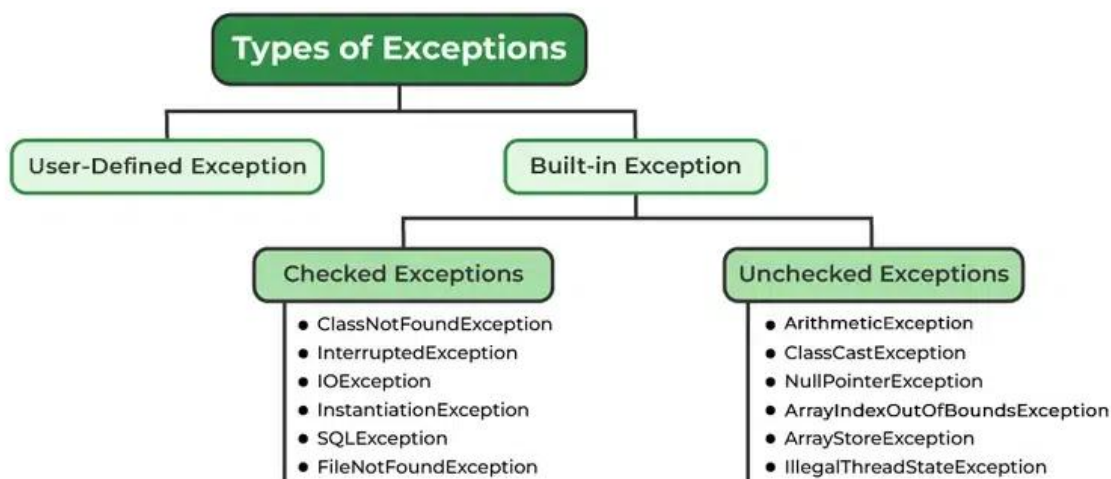
Here is a hierarchy of Java Exception classes:

- **Throwable:** Throwable is the parent class of Java Exceptions hierarchy and it has two child objects, Error and Exception. The Throwable class has two subclasses:
 - **Error:** The errors are exceptional scenarios that are out of the scope of application and it's not possible to anticipate and recover from them, for example, hardware failure, JVM crash, etc.
 - **Exception:** The exceptions are the exceptional scenarios that can be caught and recovered from, for example, FileNotFoundException, SQLException, etc.



Types of Java Exceptions

Java defines several types of exceptions that relate to its various class libraries. Java also allows users to define their exceptions.



What is Exception Handling?

Exception handling is a programming technique used to manage and respond to unexpected events (exceptions) that could disrupt the normal flow of a program. Here's what it means:

- **Detecting errors:** Exception handling involves identifying situations where things go wrong in a program, like trying to access a missing file or dividing by zero.
- **Responding safely:** Instead of letting the program crash, exception handling lets you write code to deal with these problems—often using special keywords like try, catch, or finally.
- **Maintaining flow:** With proper exception handling, your application can recover gracefully and continue running or exit cleanly, rather than causing a system-wide crash.
- **Improving code reliability:** Handling exceptions makes your software more reliable and user-friendly, as it prepares for and manages possible failures.

Exception Handling Example:

```
public class Main {  
    public static void main(String[ ] args) {  
        try {  
            int[] myNumbers = {1, 2, 3};  
            System.out.println(myNumbers[10]);  
        } catch (Exception e) {  
            System.out.println("Something went wrong.");  
        }  
    }  
}
```

Output: Something went wrong.

Java Exception Keywords

There are five keywords used to handle exceptions in Java:

- **try:** The try block is used to enclose the code that might throw an exception.
- **catch:** The catch block is used to handle the exception. It must be used after the try block only.
- **finally:** The finally block is used to execute the important code of the program.

It is executed whether an exception is handled or not.

- **throw:** The throw keyword is used to throw an exception.
- **throws:** The throws keyword is used to declare an exception.

Ways of Organizing TRY-CATCH-FINALLY:

1. try {}
catch(e) {}

2. try {}
finally {}

3. try {}
catch(e) {}
finally {}

4. try {}
catch(e) {}
catch(e) {}
catch(e) {}

5. try {}
catch(e) {}
catch(e) {}
catch(e) {}
finally {}

Here's a simple Java program that demonstrates the use of multiple catch blocks with a finally block:

```
public class MultipleCatchDemo
{
    public static void main(String[] args) \
    {
        try
        {
            int[] numbers = {10, 5, 0};
            System.out.println("Result: " + (numbers[0] / numbers[2]));
            //Division by zero
            System.out.println("Access value: " + numbers[3]);
            //Array index out of bounds
        }
        catch (ArithmeticException e)
        {
            System.out.println("Arithmetic error: " + e.getMessage());
        }
        catch (ArrayIndexOutOfBoundsException e)
        {
            System.out.println("Array index error: " + e.getMessage());
        }
        finally
        {
            System.out.println("Finally block executed. Resource cleanup
if needed.");
        }
    }
}
```

Explanation:

- The try block contains code that may cause different types of exceptions.
- There are two catch blocks:
- One handles arithmetic errors (like division by zero).
- The other handles array index errors (accessing an index that doesn't exist).
- The finally block will execute no matter what, and is used for cleanup actions.

Key Notes:

1. 'try' block cannot exist independently. It must be followed either by a catch block or a finally block
2. Whenever an exception occurs, the flow of control gets transferred to the appropriate catch block.
3. The order in which catch blocks are placed is important.
i.e sub-type catch blocks must appear prior to super-type catch blocks.
4. finally block will get executed either after a try block or a catch block. One can have several lines of executable code in a finally block.
5. try block or catch block or finally block must be enclosed within a pair of braces even if the block contains only one statement.

Thank You 😊